DAILYOFFICE · PLAYBOOK

Product Directory — User Guide

Vendor products → the right Shopify websites, priced, written and published automatically.

For the owner and staff · updated November 2026 · also in the app: Playbook → Guide

Contents

01 What the Product Directory does

02 Before you start: categories, the AI reader and vendors

03 Step 1 — Add and connect each Shopify store

04 Step 2 — Decide which products go to this website

05 Step 3 — Set the selling price

06 Step 4 — Tags in this website's structure

07 Step 5 — Titles, descriptions and SEO

08 Step 6 — Publishing mode, the Active switch and pausing

09 Store profile — one voice per store

10 Connect Telegram — vendor posts become products by themselves

11 Import products from a website

12 Tags: your own lists

13 Daily use — adding a product

14 What each status means

15 Working with listings

16 Already added? — copies, colours and where a product came from

17 Troubleshooting

18 Questions people ask

01 What the Product Directory does

One place for every product your vendors send — and a robot that puts each one on the right Shopify websites, at the right price, in the right words.

Every product a vendor sends lands in **Products**. From there the app works on its own, every minute: it reads the photos and the vendor's text, decides which of your Shopify websites the product belongs on, works out the selling price for each website, writes a title, description and Google (SEO) text in that website's own style, adds that website's tags, and puts the product on the store.

How one product travels

1. **Lands** — a product comes in. It comes straight from the vendor's Telegram channel (see **Connect Telegram**), or you add it in **Products → + Add product**. Status: **New**.
2. **Read** — the AI looks at the photos and the vendor's post and fills in name, category, fabric, colour, sizes, cost price and other details (work, occasion, wash care...). Status: **Reading**, then **Ready** — or **Needs info** when the cost price or the category is missing.
3. **Sorted** — each website's **Routing rules** decide whether the product goes there. Every "yes" becomes a *listing*: one product on one website.
4. **Priced and written** — for each listing the price is worked out from the cost with that website's formula, and that website's AI writer writes the title, description, SEO text, tags and photo descriptions.
5. **Published** — the listing is sent to Shopify: live, as a draft, or held until someone approves it — whatever that website's **Publishing mode** says.

> **Tip** One product can be on several websites, each with its own price, title, description and tags. Changing one website never changes another.

Where things are

PAGE	WHAT IT IS FOR
Dashboard	What needs a person today, products by status, and each website's numbers (in progress, needs review, published, failed). Tap any tile to see those products.
Products	Every product. Add new ones, search and filter, open one to see its facts and its listing on each website.
Websites	Your Shopify stores. Each has six tabs: **Connection**, **Publishing**, **Routing rules**, **Pricing**, **Tags**, **Content & AI**.
Vendors	The people you buy from — name, a short code and their Telegram channel.
Settings	The AI that reads products, the category list, **Telegram** (bot, admin group, crawler), and the switch that pauses all automation (plus **Run now**).
Guide	This guide, with **Download PDF**.

Who can do what

- Admins, managers and operators can use every page: add and edit products, approve, publish, change website settings.
- Only an **admin or manager** can save a store's credentials (the Shopify keys), the Telegram bot token and the crawler key.

02 Before you start: categories, the AI reader and vendors

Ten minutes, once. Everything after this depends on it.

Categories

1. Open **Settings → Categories & tags**.
2. Under **Categories**, type each category you sell and tap **Add** — for example `Saree`, `Kurta Set`, `Lehenga`, `Dupatta`, `Co-ord Set`.
3. Tap a category to choose the attributes it needs (Colour, Work…) — see **Tags: your own lists**.

The AI must pick one of these for every product. A product without a category — or without a value for one of its category's attributes — waits in **Needs info**. Routing rules, tags and the Shopify product type all use the category, so spell each one exactly the way you want it to appear on your websites.

The AI that reads products

1. In **Settings → AI that reads products**, pick a **Model** and an **Effort**. `Haiku 5.5` with `Model default` is enough for reading.
2. Tap **Save reader**.

This AI only reads (fills in the facts). Each website chooses its own writer later, under **Websites → Content & AI**.

Vendors

1. Open **Vendors → + Add vendor**.
2. **Name**: the vendor as you know them, e.g. `Rangoli Creations`.
3. **Short code** (optional): two or three letters, e.g. `RG`. Tags can use it as `{vendor_code}`.
4. Tap **Save vendor**. Repeat for each vendor.

> **Tip** Stopped buying from someone? Tap **Deactivate** instead of deleting. Their old products stay; they just stop showing up as a choice for new ones.

03 Step 1 — Add and connect each Shopify store

Do this once for every website. You need the store's myshopify.com address and a Shopify login

that is allowed to install apps (the store owner, or staff with that permission).

Add the website

1. Open **Websites → + Add website**.
2. **Name**: how you call it, e.g. `Indya Ethnic`.
3. **Store domain**: the myshopify.com address, e.g. `indya-ethnic.myshopify.com` — find it in Shopify admin → Settings → Domains. Not your own .in / .com domain.
4. Tap **Add website**. The website opens on its **Connection** tab.

A new website starts **Off** with the routing set to **Do not list it**, so nothing is sent until you have finished setting it up.

Which way to connect?

YOUR SITUATION	WHAT YOU USE	IN THE APP PICK
The store already has an older custom app made inside the Shopify admin, with an Admin API access token that starts `shpat_`	That app's access token	**Access token**
Anything else — a new store, or no such app	A new app from the Shopify Dev Dashboard	**Client ID + secret**

Note Shopify stopped letting anyone create new "legacy" custom apps in the store admin on 1 January 2026. Old ones keep working. Every new connection is made in the Shopify Dev Dashboard (Way B).

Way A — an existing custom app (token starting shpat_)

1. In the Shopify admin open Settings → Apps and sales channels → Develop apps, and open the app.
2. Check its Admin API scopes include `write_products`, `read_products`, `write_publications`, `read_publications` — plus `write_inventory` if this website will track stock. If you add any, save and reinstall the app when Shopify asks.
3. Copy the **Admin API access token** (it starts with `shpat_`). Shopify shows it only once — if nobody saved it, use Way B instead.
4. In the app: **Websites →** your website **→ Connection**. Under **How the app signs in** pick **Access token**, paste it into **Admin API access token**, tap **Save credentials**.

Way B — a new app from the Shopify Dev Dashboard

1. Sign in to the Shopify Dev Dashboard (dev.shopify.com) with a Shopify account in the **same organisation as the store**. An app from another organisation cannot connect to the store.
2. Create an app, e.g. `DailyOffice Product Directory`.

3. In the app's access settings add the scopes `write_products`, `read_products`, `write_publications`, `read_publications` — plus `write_inventory` if this website will track stock. Release that version.
4. Install the app on the store and approve it.
5. Open the app's settings and copy the **Client ID** and the **Client secret**.
6. In the app: **Websites** → your website → **Connection**. Under **How the app signs in** pick **Client ID + secret**, paste both, tap **Save credentials**.

> **Tip** Shopify renames its screens now and then. If a label above looks different, follow the same idea: create app → give permissions (scopes) → release → install on the store → copy client ID and secret.

Test, then switch on

1. Tap **Test connection**.
2. Green **Connected to <store name>. All needed permissions are there.** — done.
3. Yellow **Missing permissions: …** — add those scopes to the app in Shopify, reinstall / approve it, then **Test connection** again.
4. Red **Last problem: …** — the details are wrong or the app was removed. Copy them again, **Save credentials**, test again.
5. Finish Steps 2 to 6 below for this website. Then come back to **Connection** and switch **Active** on.

- **Active** can only be switched on after a successful connection. Switching it on also sends the Ready products already in the directory that match this website's rules. Off = nothing new is sent to this store; products already there stay.
- Credentials are kept on the server only and are never shown again — the boxes stay empty. Leave them empty to keep what is saved; type only to replace.
- With **Client ID + secret** the app renews its Shopify access by itself. Nothing to do.
- Moving a website to a different Shopify store: change **Store domain**, enter that store's credentials and tap **Save credentials**. The website is switched off, the old store's keys are deleted, and its products are queued to be sent again to the new store. Products already on the old store stay there.

04 Step 2 — Decide which products go to this website

Websites → your website → Routing rules.

When no rule matches

CHOICE	WHAT IT MEANS
Do not list it (safe default)	Nothing goes to this store unless a rule sends it. Recommended.

CHOICE	WHAT IT MEANS
List everything	Every product goes to this store unless a rule keeps it off.

This choice is saved with **Save changes** at the bottom of the screen. Rules save on their own.

Add a rule

1. Tap **+ Add rule**.
2. **Rule name** — for you, e.g. `Silk sarees from Rangoli` or `Nothing under ₹500`.
3. **When a product matches** — **Send it to this store** or **Keep it off this store**.
4. Fill only the conditions you need. Every condition you fill must match; empty ones are ignored: **Vendors**, **Categories**, **Fabrics** (any of these, capital letters do not matter), **Cost from ₹** / **Cost up to ₹** (the vendor cost), **Keywords** (any of these words in the name, SKU, details or what the AI read).
5. Optional, for "Send" rules: switch on **Own price for these products** and set a **Markup %** and **Add ₹** that replace the website's own for these products.
6. Tap **Save rule**.

Order matters

Rules are checked from the top. The **first** switched-on rule that matches decides, and the rules below it are not looked at. Use the ↑ ↓ buttons to change the order. Put "Keep it off" rules above wide "Send" rules.

#	RULE	SHOWN AS
1	Nothing under ₹300	**KEEP OFF** · cost ₹0–₹300
2	Bridal lehengas	**SEND** · Lehenga · words: bridal · own price: +150% + ₹200
3	Silk sarees from Rangoli	**SEND** · vendor Rangoli · Saree · fabric Silk

With "When no rule matches" set to **Do not list it**: a ₹250 silk saree from Rangoli is stopped by rule 1. A ₹1,800 silk saree from Rangoli is sent by rule 3. A cotton kurta matches no rule, so it is not listed.

- A rule with no conditions matches every product (the app warns you).
- The switch on a rule turns it off without deleting it.
- A change applies to products that arrive or are checked again after it. For the products already in the directory, tap **Send matching products now** under the rules (or open one product and tap **Check websites again**).
- Deleting a rule never removes products that are already listed.

05 Step 3 — Set the selling price

Websites → your website → Pricing. The price in the vendor's post is your COST; each website turns it into its own selling price.

Selling price = cost × (1 + Markup % ÷ 100) + Add ₹, then rounded, and never below the minimum price.

BOX	WHAT TO TYPE
Markup %	`100` doubles the cost, `80` adds 80%.
Add ₹ (after markup)	A fixed amount on top — shipping, packing, payment charges.
Rounding	How the final number looks (table below).
Compare-at % (was price)	`0` = no struck-out price. `30` = show a "was" price 30% higher.
Minimum price ₹	Never sell below this. Empty = no minimum.

Worked example: cost ₹640, Markup 80%, Add ₹60

640 × 1.8 = 1,152, plus 60 = **₹1,212** before rounding.

ROUNDING	HOW IT WORKS	₹1,212 BECOMES
No rounding	Kept as it is (to the paisa)	₹1,212
Nearest 10	To the nearest ten	₹1,210
Ends in 99	Up to the next ...99. A price already ending in 99 stays (₹1,299 stays; ₹1,300 → ₹1,399)	₹1,299
Ends in 49 / 99	Up to the next ...49 or ...99	₹1,249

Compare-at (the struck-out "was" price)

With **Ends in 99** and **Compare-at %** `30`: 1,299 × 1.3 = 1,688.70, rounded the same way → shoppers see ₹1,699 struck out next to **₹1,299**.

Minimum price

Cost ₹250: 250 × 1.8 + 60 = 510 → Ends in 99 → ₹599. With **Minimum price ₹** `799` the website sells it at **₹799**. The minimum is applied after rounding, so choose a "nice" number like 799 or 999.

A rule's own price

The "Bridal lehengas" rule above uses +150% + ₹200. Cost ₹6,000: 6,000 × 2.5 + 200 = 15,200 → Ends in 99 → **₹15,299**. Rounding, compare-at and the minimum still come from the Pricing tab.

> **Tip** Use the **Preview** box: type a **Sample cost ₹** to see the price, the struck-out price and your margin before you tap **Save changes**.

New pricing applies to listings written from now on. A listing already on Shopify keeps its price until its product is checked again: open the product and tap **Check websites again** — the new price goes to Shopify and the text stays. A price typed by hand (**kept**) never changes on its own.

06 Step 4 — Tags in this website's structure

Websites → your website → Tags. Each store filters and builds collections from tags in its own way — one may use "Fabric_Silk", another plain "silk". This tab sets the pattern for this store.

- **Tag templates** — each line becomes one tag. Type it and press Enter. Words in curly brackets are filled from the product.
- **Fixed tags** — added to every product sent to this store, e.g. `New Arrival`.
- **How {tags} is written** — **Plain** sends the value as it is (`Maroon`); **Prefixed** puts the attribute in front (`Colour_Maroon`, or the attribute's own Shopify prefix from the Tags page).
- **Letter case** — **As written**, **lowercase** or **Title Case**.
- **AI extra tags** — switch on to let the AI add style / occasion tags it sees in the photos and text, up to **At most** tags. Fill **Only from these tags** and it picks from that list and adds nothing else.
- **How many tags** — **At most, per product** caps the final list (8 on a new website), after the templates, fixed tags and AI extras.

PLACEHOLDER	FILLED WITH	EXAMPLE
`{tags}`	The product's chosen values (see **Tags: your own lists**), one tag each, plain or prefixed — new websites start with it	Maroon, Zari — or Colour_Maroon
`{category}`	The category	Saree
`{fabric}`	The fabric	Banarasi Silk
`{color}`	The colour	Maroon
`{vendor}`	Vendor name	Rangoli
`{vendor_code}`	Vendor short code	RG
`{size}`	One tag per size	Size_S, Size_M, Size_L
`{sku}`	Product code	RG-101
`{attr:occasion}`	Any detail the AI read — work, occasion, pattern, set_contents, wash_care…	Festive
`{attr:store_tags}`	The other store's tags, raw (Import from website) — usually `{tags}` is what you want	Ideal Collection

Example

Templates `{category}`, `Fabric_{fabric}`, `Color_{color}`, `Size_{size}`, `Vendor_{vendor_code}`, `Occasion_{attr:occasion}` and the fixed tag `New Arrival`, for a maroon Banarasi silk saree from Rangoli (RG), Free Size, festive, give:

`Saree` `Fabric_Banarasi Silk` `Color_Maroon` `Size_Free Size` `Vendor_RG` `Occasion_Festive` `New Arrival`

- If a placeholder is empty for a product, that tag is skipped — you never get a half tag like "Fabric_".
- Commas inside a tag become spaces (Shopify splits tags at commas). Repeats are removed, whatever the capital letters.
- The **Example** box at the bottom of the tab shows the tags for a sample product as you type.
- Shopify's product type is set to the category automatically.

07 Step 5 — Titles, descriptions and SEO

Websites → your website → Content & AI. For every listing the AI writes the title, description, SEO title, SEO description, web address (handle), two questions and answers, extra tags and photo descriptions — for this website alone (see **Store profile**).

Model and effort

MODEL	GOOD FOR	COST
Haiku 5.5	Fast; good for most listings	Lowest
Sonnet 5.5	Better writing — your main store	About 20 × Haiku
Opus 5.5	The best writing — a premium store	Highest

Effort (**Model default**, **Low**, **Medium**, **High**, **Extra high**, **Max**) is how hard the model thinks before writing. Higher is slower and costs more. **Model default** suits most listings; models that do not support effort ignore it.

> **Tip** Start every website on Haiku 5.5 with Model default. Read 10–20 listings. Move a website up to Sonnet only if its writing is not good enough.

The other settings

SETTING	WHAT IT DOES
Language	`English`, `Hinglish`, `Hindi`...
Brand voice	How this store talks, e.g. "premium, elegant, festive — for brides and wedding guests".
Image alt text	On: the AI describes each photo for Google and for screen readers.

SETTING	WHAT IT DOES
Title pattern	`{ai_title}` lets the AI write the title. Or build it: `{color} {fabric} {category}` → "Maroon Banarasi Silk Saree". Also usable: `{vendor}`, `{sku}`, `{name}`; mix them, e.g. `{ai_title} {sku}`.
Title style for the AI	How the AI should build its title, in your words, e.g. "Colour, fabric, type, then the work". The search phrase from the store profile is always in it.
Length limits	**Title (characters)**, **Description (words)** (200 on a new website: the AI writes about two thirds of it or more), **SEO title (characters)** (Google shows about 60), **SEO description (characters)** (about 155).
Extra instructions	Anything to always do or avoid, e.g. "Mention free shipping above ₹1,999. Never use the word cheap."

Note The AI is told to use only the facts given and never to invent a fabric or size. Still, read the first listings of every new website before you set it to Live.

Plain, natural wording

The writer describes a product the way someone who knows it would tell a shopper: plain facts, no sales pitch. It is told to leave out the usual AI habits, such as slogans ("Elegance redefined."), "not just a kurta, it's a statement", praise words like *stunning* or *exquisite*, benefits nobody stated ("ensuring all-day comfort") and long dashes. When there is little to say about a product it writes less instead of padding. The rules follow the Humanizer writing guide.

Tip **Brand voice** and **Extra instructions** still have the last word. If a store should sound more festive or more like an advert, say so there.

Descriptions use only simple formatting (paragraphs, bullet points, small headings, bold, italics, line breaks), so they look right on any Shopify theme. Save the tab with **Save changes**.

08 Step 6 — Publishing mode, the Active switch and pausing

Websites → your website → Publishing.

PUBLISHING MODE	WHAT HAPPENS	USE IT WHEN
Live	Published and visible on the store straight away	You trust this website's setup
Draft	Sent to Shopify as a draft — someone makes it visible inside Shopify	You want a last look inside Shopify
Review first	Waits in Products as **Needs review** until someone taps **Approve**, then publishes	A new website, the first weeks

Track stock — on: Shopify counts stock and stops selling at 0 (the app needs `write_inventory`). New products start at 0, so they cannot be bought until you set their quantities in Shopify. Off: always orderable — right for made-to-order or vendor-held stock.

Three ways to stop things

SWITCH	WHERE	WHAT IT DOES
Active (one website)	Websites → website → Connection	Off = nothing new is sent to this store. Products already there stay. On again = the matching products are sent.
Rule switch (one rule)	Websites → website → Routing rules	That rule stops sorting products.
Automation (everything)	Settings → Automation	Paused = nothing is read, written or published on its own. Buttons like **Publish now** still work. The Dashboard shows a yellow banner.

Settings → Run now does one round straight away (it can take up to 2 minutes) and shows how many products were read, routed, written, published and failed.

A safe first day for a new website

1. Set **Publishing mode** to **Review first** and tap **Save changes**.
2. On **Connection**, switch **Active** on.
3. Add three or four real products in **Products**.
4. Open each one, open this website's listing, read it, fix anything, tap **Approve**.
5. Check the products on the store. Happy? Switch **Publishing mode** to **Live**.

09 Store profile — one voice per store

Websites → your website → Store profile. Fill it once for each store. The AI reads it before every listing, so two stores get a different title, description, SEO text, questions and photo descriptions for the same product. The facts (fabric, colour, work, sizes) stay the same everywhere.

SETTING	WHAT IT DOES
Brand name	Every SEO title ends with it, e.g. "... | Kreeva". Empty uses the website's name.
Market	Where the shoppers are (India, USA, UK...). The writer uses the words they search with.
Spelling and **Measurements**	**British / Indian** (colour, jewellery) or **American** (color, jewelry); **Inches** or **Centimetres**.
Audience	Who shops here, in a line.

SETTING	WHAT IT DOES
Angle	What this store leads with. The description opens with it. Give each store a different one.
Search phrase	The phrase each listing is written for, built from the product: `{color} {fabric} {category}` gives "Maroon Georgette Kurta Set". Also usable: `{work}`, `{occasion}`, `{name}`.
Questions in the description	On: the two questions and answers sit at the end of the description, so every theme shows them.
Custom fields	On: fabric, work, colour, occasion, what is in the set and the questions also go to Shopify as custom fields (`custom.fabric`, `custom.faq` ...) for themes and filters that read them. Other apps' fields are never touched.
Ordering details	**Delivery time**, **Stitching** and **Returns**. The writer puts them in an "Ordering from ..." part of the description, in its own words. Leave all three empty and that part is left out.

What every listing gets

- A **title** with the search phrase in it once.
- A **web address** of 3 to 6 plain words, with no design codes.
- An **SEO title** that ends with the brand name, and an **SEO description** of about 140 to 155 characters.
- A **description**: an opening in the store's angle, 5 to 7 fact bullets, "How to wear it", and "Ordering from ..." when you gave ordering details.
- **Two questions and answers** a shopper would really ask, answered only from the facts and your profile.
- A different **description for each photo** (alt text), and tags from your own lists.

Checked before it goes out

After the AI writes a listing, a program checks it. If something is wrong the AI rewrites it once. If it is still wrong, the listing waits in **Needs review** with the reason, even on a **Live** website, and you decide.

THE CHECK	WHAT IT CATCHES
Copied wording	Five or more words in a row taken from the supplier's text or from another store's version. Facts such as "Kurta, Pant and Dupatta" may repeat.
Same as another store	The same title, SEO title or opening line as the listing on another store.
Search phrase	Missing from the title, from the first 50 words or from the SEO title.
Names and claims	The supplier's name, a design code, a link or phone number; "No. 1", "only 3 left", invented reviews.

THE CHECK	WHAT IT CATCHES
Ordering details	A number of days that is not in your store profile.

Open the listing to see the reason and the smaller notes ("Also worth a look"). You can fix the text by hand, tap **Regenerate**, or **Approve** it as it is. The editor also shows the **Search phrase** (change it, save, then **Regenerate**) and the two **Questions**.

> **Note** Nothing is written for a product until its necessary details are there: cost price, category, fabric, colour, at least one photo and its required tags. Until then it stays in **Needs info** and says what is missing.

What the AI cost

Each website's listing shows **AI cost** in rupees. The product page adds everything up for that product: reading it, choosing tags and writing for every website, rewrites included. The Dashboard shows the total for the month. Dollars are turned into rupees at ₹95.

> **Tip** The first store written for a product has nothing to differ from. Each later store is told what the earlier ones say and must open differently, so the order does not matter to you.

10 Connect Telegram — vendor posts become products by themselves

Do this once (about an hour). After it, every product a vendor posts in their Telegram channel is in Products the next morning, with the vendor already filled in — taken from the channel, because the post itself never names the vendor.

How it works

1. A **crawler** — one of your staff Telegram accounts, running on a small cloud server — looks at each vendor channel **once a day**, overnight, each channel at its own time.
2. It **forwards** that channel's new posts into your private **admin group**, one channel at a time, so the group is never flooded.
3. The **Playbook bot** is an admin in that group. It passes every forwarded message to Playbook.
4. Playbook joins the photos, videos and text of one post into one product and sets the vendor from the channel. Then the usual steps run: read by the AI, sorted, priced, written, published.

You need: your own Telegram account (to make the bot), a **staff Telegram account** for the crawler (a normal account with a phone number, not a bot — bots cannot read other people's channels), a laptop for one sign-in, a GitHub account that can see **kreeva-lifestyle/unsort** (an owner of that organisation approves Railway's access), and a card for Railway's **Hobby** plan (about US$5 a month).

1 · Create the bot in @BotFather

1. In Telegram search for **@BotFather** (blue tick) and open it.
2. Send `/newbot`. Give it a name, e.g. `DailyOffice Playbook`, then a username that ends in `bot`, e.g. `dailyoffice_playbook_bot`.
3. BotFather sends a **token** — numbers, a colon, then letters, e.g. `123456789:AAH…`. Tap it to copy it. Keep it private.

> **Tip** Come back to @BotFather after step 3 (once the bot is in your group): send `/setjoingroups`, pick your bot and choose **Disable**. Then nobody can add the bot to other groups.

2 · Save the token in Playbook

1. In Playbook open **Settings → Telegram**. Under **1 · Playbook bot**, paste the token into **Bot token** and tap **Save token**.
2. You see the bot's @name and **Connected**. If it says **Telegram can't reach Playbook**, tap **Check bot**; if it stays red, tap **Change token** and save the same token again.

- Only an **admin or manager** can save the token. Everyone else sees the status.
- The token is kept on the server and never shown again.

3 · Add the bot to your admin group and link it

1. In Telegram make a **private group**, e.g. `Vendor posts`, with you and the **staff account** (step 4). Keep it just for this.
2. Open the group, tap its name → **Edit** (pencil) → **Chat history for new members** → **Visible** → save. This turns it into a supergroup, the only kind Playbook can use. If you skip it, the bot tells you so when you send the link code.
3. Open the group, tap its name → **Add members** → add your bot. Then tap **Edit** (pencil) → **Administrators** → **Add Admin** → pick the bot → tap ✓ to save. The bot must be an **admin** to see the forwarded posts.
4. In Playbook, under **2 · Admin group**, tap **Get link code**. A 6-digit code appears, valid for 30 minutes.
5. In the group send `/link 123456` (your code instead of 123456) — or tap **Copy /link …** in Playbook and paste it.
6. The bot answers **Linked to Playbook ✓**, and Settings shows **Linked** with the group's name within a few seconds.
7. Now go back to **@BotFather** → `/setjoingroups` → your bot → **Disable** (see the tip in step 1).

> **Tip** Messages people type in the group are ignored; only forwarded vendor posts become products. 5 wrong codes and the code stops working — tap **New code**.

4 · The staff account: its keys, and join every vendor channel

1. On a laptop open **my.telegram.org** and sign in with the **staff account's** phone number (the code arrives in that account's Telegram app).
2. Open **API development tools**. Create an app — any name, e.g. `Playbook crawler`, platform **Other**.
3. Copy the **api_id** (a number) and the **api_hash** (letters and numbers). You need both in step 6.
4. From the staff account, **join every vendor channel** you want to read, and make sure it is a member of the admin group and can send messages there.

> **Tip** In the staff account: **Settings → Privacy and Security → Groups & Channels → My Contacts**, so strangers cannot pull it into their groups.

5 · Create the crawler key

1. In Playbook, under **3 · Crawler**, tap **Create crawler key** (admins and managers only).
2. Tap **Copy key** and paste it into Railway (step 6). It is **shown only once** — when it is saved, tap **I saved it — hide**.

> **Tip** Best done when Railway's **Variables** page is open (step 6), so you paste the key straight in. Lost it? Tap **Replace crawler key** — nothing uses it until the crawler runs.

Replace crawler key makes a new key and stops the old one at once — paste the new one into `PB_CRAWLER_KEY` on Railway straight away.

6 · Start the crawler on Railway

First make **TG_SESSION** — the staff account's sign-in for the crawler. Do this once, on a laptop:

1. Install **Node.js** from **nodejs.org** — the big **LTS** button (the one for most users) — and click through the installer.
2. Download the crawler: on GitHub (signed in) open **kreeva-lifestyle/unsort** → green **Code** button → **Download ZIP**, and unzip it.
3. Open the folder `unsort-main/tools/pb-telegram-crawler` in a terminal. Windows: right-click the folder → **Open in Terminal**. Mac: open **Terminal**, type `cd` (with a space), drag the folder into the window, press Enter.
4. Type `npm install` and press Enter. Wait until it finishes. Windows: if it says *running scripts is disabled*, type `npm.cmd install` instead (and `npm.cmd run login` below).
5. Type `npm run login` and press Enter. Type the **api_id**, the **api_hash**, the **staff phone number** with +91, then the **code Telegram sends in the app**, and the two-step password if the account has one. Nothing shows while you paste the api_hash or type the password — that is normal.
6. It prints **TG_SESSION** once — a long line of letters. Copy it for the Variables below.

Note **TG_SESSION is a login.** Anyone who has it can read and send messages as the staff account. Paste it only into Railway's Variables — never into a chat, email or WhatsApp. To cancel it: Telegram on the staff phone → **Settings → Devices** → end that session, then run `npm run login` again.

Then put the crawler on Railway:

1. Open **railway.com** → **Sign up** with GitHub → choose the **Hobby** plan and add a card.
2. **New Project → Deploy from GitHub repo**. When Railway asks, allow it to see the **kreeva-lifestyle** organisation, repository **unsort** — then pick **unsort**.
3. Railway starts a first build straight away — it fails (red). That is expected; it works once the Root Directory is set below.
4. Open the new service → **Settings**. Set **Root Directory** to `tools/pb-telegram-crawler`.
5. In the same **Settings**: **Region** → **Southeast Asia (Singapore)**; **Restart Policy** → **Always**; **Replicas** → 1.
6. Open **Variables** → **+ New Variable**. Type the name exactly as below, paste the value, tap **Add**. Do it for each line:

NAME	VALUE
`TG_API_ID`	api_id (step 4)
`TG_API_HASH`	api_hash (step 4)
`TG_SESSION`	the long line from the laptop
`PB_URL`	`https://ulphprdnswznfztawbvg.supabase.co/functions/v1/pb-telegram`
`PB_CRAWLER_KEY`	the key from step 5
`RAILWAY_DEPLOYMENT_OVERLAP_SECONDS`	`0`
`RAILWAY_DEPLOYMENT_DRAINING_SECONDS`	`15`

1. Tap **Deploy**. In **Deploy Logs** you see `Connected to Telegram`.
2. Within a minute, Playbook **Settings → Telegram → 3 · Crawler** shows **Running** and **Last seen just now**.

- Railway's **Hobby** plan, about **US$5 a month**, is enough. The crawler runs all day but only reads Telegram at each channel's time.
- Run only **one** crawler. Two copies at once can make Telegram end the staff account's session and would forward posts twice. Never run `npm start` on the laptop.
- Any Linux server with Docker works instead of Railway — the steps are in the crawler's README.

7 · Add each vendor's channel

1. Open **Vendors → Edit** on a vendor (or **Add vendor** — the **+** at the top on a phone).
2. In **Telegram channel** type the channel name — `@kumari_art`, `kumari_art` or a link like `t.me/kumari_art` all work. Tap **Save vendor**.
3. Playbook gives the channel its **own daily time** by itself. The vendor row shows it, e.g. `@kumari_art · checked daily at 1:12 AM`, and **Not checked yet** until its first night.
4. Within a few minutes the crawler notes the channel's newest post (Settings shows **Getting ready** meanwhile). Older posts are never copied — only what is posted from then on.

How the times are picked: channels are checked once a day inside the **window**, 11:00 PM to 7:00 AM unless you change it. The first channel gets the middle of the window (3:00 AM). Each new channel gets the middle of the **biggest free gap** between the others, so no two channels are checked at the same time and the admin group never gets every vendor at once.

- **Settings → Telegram → 3 · Crawler → Daily checks** lists every channel in time order: vendor, time, **Next check** and the last result.
- **Edit window** (admins and managers) changes the start and end — at least 1 hour; it may run past midnight. Saving spreads every channel evenly over the new window.
- **Spread evenly** (admins and managers) gives every channel a new time, evenly spaced, in the same order. Use it after adding or removing many vendors. Nothing is copied twice. Best done in the daytime: done at night, a channel whose new time has already passed is checked the next night.
- A vendor added in the daytime is first checked at its time that night. Clearing the channel, or **Deactivate** on the vendor, stops its checks.
- A post from a channel that no vendor has yet still comes in (if someone forwards it): Playbook makes a new vendor named after the channel. When a vendor reposts another channel's post, it still goes to the vendor whose channel it came from.

8 · How to tell it works

1. In **Settings → Telegram**: **1 · Playbook bot** says **Connected**, **2 · Admin group** says **Linked**, **3 · Crawler** says **Running** with **Last seen** a minute or two ago — all day, even though it reads Telegram only at night.
2. The next morning, in **Daily checks**, each channel shows **Last checked** a few hours ago and its **Next check** tonight.
3. In the admin group you see the night's posts, one vendor after another.
4. In **Products** the new products are there, each marked **From Telegram · <channel> · Open post** under its name, with the vendor filled in. **Messages waiting** in Settings goes back to 0 a few minutes after a channel's check ends.

Tip Want to test it the same day? Wait until the channel no longer says **Getting ready**, ask the vendor to post, then **Edit window**: Start = a few minutes from now, End = 1 hour later, **Save window**. **Daily checks** shows each channel's new time. Afterwards set it back to **11:00 PM** and **7:00 AM** and save again.

What happens to a vendor post

IN THE CHANNEL	IN PLAYBOOK
Albums of photos (up to 10 each)	Up to 20 photos on one product, in the order posted
A video up to 20 MB	Copied next to the photos (with a play button). Videos never go to Shopify or the AI
A bigger video	Skipped, with a note on the product: "Video (91 MB) too big to copy — add it by hand"
The long text (CODE, SIZE, COLOUR, RATE…), or a photo caption	The product's **Details** — the AI reads it
A short message like `200 Less` after the rate (also `Less: 200`, `Less Rs 200`, or `RATE 1600 LESS 200` on one line)	Kept as the **Less**: cost = rate − less (₹1,600 − ₹200 = ₹1,400), shown in **Facts**
A pointing-hand sticker or emoji between products	Treated as a separator and ignored
Photos with no text	A product with a note "No description arrived from Telegram". If the text comes later (within 2 days), it is added and the product is read again
The text first, then its photos	Also one product — each text goes with the photos after it
A whole day of posts at once	Split into products by the order they were posted, exactly as if they had come one by one
The same post copied twice	Counted once

Anything to check shows in a yellow **Notes from Telegram** tile on the product. A post the vendor edits after it was copied (for example a corrected RATE) is not copied again — fix the product in Playbook.

9 · When something is wrong

SETTINGS SHOWS	WHAT TO DO
Telegram can't reach Playbook on the bot	Tap **Check bot**. Still red: **Change token** and save the same token again
Waiting for /link does not change	The bot must be an **admin** in the group, and the code under 30 minutes old — tap **New code** and send it again

SETTINGS SHOWS	WHAT TO DO
Crawler **Stopped** / "No report for more than 5 minutes"	The crawler is not seen. Open Railway → the service → **Deploy Logs**, then **Restart**. Check the five Variables are there and Restart Policy is **Always**
"Waiting for the admin group — link it..."	Do **2 · Admin group** first. The crawler waits and copies nothing
"The staff account is not in the admin group"	Add the staff account to the group (step 3)
"The staff account can't post in the admin group"	In the group: its name → **Edit** → **Permissions** → turn on **Send messages** and **Send media**
"Join this channel from the staff account"	Open the channel in the **staff account** and tap **Join**. Done the same night, it is picked up within a few minutes; otherwise at its next time
"@... is a group, not a channel"	Only channels can be read — ask the vendor for their channel name
"No Telegram channel called @..."	Fix the channel name on the vendor (**Vendors → Edit**)
"This channel blocks forwarding"	The vendor turned on "Restrict saving content". Those posts are skipped; ask them to turn it off, and add those products by hand
"Could not forward — check the staff account is still in the admin group"	Add the staff account back to the group. Nothing is skipped — the check carries on from the same post next time
"Telegram asked us to slow down for N min"	Telegram's speed limit. Nothing to do — the crawler waits N minutes and carries on where it stopped
"Lost the connection to Telegram — reconnecting"	Nothing to do — it reconnects by itself. Still there after an hour: **Restart** on Railway
"Telegram error: ..."	**Restart** on Railway; if it comes back, tell the developer and send the exact text
A channel was not checked on the night you changed the window or tapped **Spread evenly**	Nothing is lost — its posts come at its new time the next night
"Telegram signed the staff account out"	Run `npm run login` again on the laptop (step 6) and replace **TG_SESSION** on Railway
The staff account is **limited** (it cannot message, or Telegram says it is restricted)	Stop the crawler on Railway. Email **recover@telegram.org** from any address: say it is your staff account reading your suppliers' channels for your shop, with its phone number. Start it again once the limit is lifted
"... being tried again" next to Messages waiting	Nothing to do — Playbook tries those messages again by itself (up to 3 times)

SETTINGS SHOWS	WHAT TO DO
"... could not be used" next to Messages waiting	Open **See which posts**, tap **Open post** and add that product by hand; tell the developer if it keeps happening

11 Import products from a website

Paste another store's web address and Playbook copies its in-stock products into **Products**. From there the usual steps run: the AI reads each one, the website rules decide where it goes, and the listing is written and published.

How to do it

1. Open **Products**. On a computer tap **Import from website** next to **+ Add product**; on a phone tap the **Import from website** row at the top of the list.
2. Type or paste the store's address (for example `example.com`) and tap **Check website**. Playbook looks at the store for a few seconds. The whole store is imported, even if you paste the address of one collection.
3. Read the preview: what kind of store it is, about how many products it has, three sample products with their prices, and anything to know (in the yellow box).
4. **Vendor** — who these products come from. A website does not say who made a product, so Playbook asks once per website. The box starts from the store's name; tap one of your vendors shown under it, or type a new name. Every imported product gets this vendor. A website you imported before is **not asked again**: its vendor is shown with **Change**.
5. Leave **Only in-stock products** on unless you also want sold-out items.
6. Leave **Sell at their price** on to use the store's price as our selling price (see below).
7. Tap **Import about 980 products** (the number is the estimate; it reads **Import up to ...** when the store cannot say how many are in stock). A tile at the top of **Products** shows the progress; you can leave the page, it keeps going on the server.

What Playbook can read

KIND OF STORE	WHAT YOU SEE	WHAT IS COPIED
Shopify store	Green dot	Every product: name, description, photos (up to 10), sizes, colour, price, whether it is in stock
WooCommerce store	Green dot	The same, from the store's own product list
Other store — best effort	Yellow dot	Only the pages that carry product data, found through the site map. Some products may be missed, and it is slower (about 25 pages a minute)

KIND OF STORE	WHAT YOU SEE	WHAT IS COPIED
Unknown store	Red box	Nothing was found — check the address or try the store's home page

"In stock" means the store itself says the product (any of its sizes) can be bought right now. A product whose every size is sold out is left out while **Only in-stock products** is on.

The details it copies

Besides the name, description, photos, sizes and price, Playbook keeps every detail the other store shows about a product. They appear on the product page in the **From the website** tile (text only — they are never sent back to that store), and the AI reader leaves them as they are.

ON THE PRODUCT PAGE	WHERE IT COMES FROM
Uploaded on their site, **On sale there since**	The dates the other store keeps for the product: when it was added there, and — only when it is a different day — since when shoppers could see it. Shopify stores give both; other stores usually give none
Brand	The store's brand or vendor field
Manufacturer	Only when the store names a maker separately from the brand
Product type	The store's product type or categories
Material, **Colour**	The store's Material / Fabric and Colour options, every value it lists
Store tags	The store's own tags, comma-separated
SKU, **Barcode**, **Weight**	The first size's codes and weight. The SKU also fills the product's SKU when that is empty
Was price, **Currency**	The crossed-out "was" price of the size that sets the price, and the store's currency (shown only when it is not rupees)
Page title, **Page description**	The product page's own title and description, as search engines see it

In a tag template (**Websites → Tags**) write `{attr:brand}`, `{attr:manufacturer}`, `{attr:product_type}`, `{attr:sku}` or `{attr:barcode}` for one tag, and `{attr:store_tags}` for one tag per store tag.

A detail the store does not show is simply left out — nothing is guessed. **Open page** at the top of the product page opens it on the other store. To tag every imported product with its brand, add the template `Brand_{attr:brand}` under **Websites → Tags**; a product without a brand just gets no brand tag.

The price

With **Sell at their price** on, the price shown on the other store becomes **our selling price on every website** — the lowest price of the sizes that are in stock. The websites' pricing rules (markup, rounding, minimum) are skipped for these products, and the product's **Facts** say "Sells at ₹... on every website (price copied from ...)". A cost price is optional for them — they become **Ready** without one. Switch it off and no price is copied: each product waits in **Needs info** until you type a cost price, and the websites' pricing rules apply as usual.

Prices in another currency are never copied (a $29 dress must not become ₹29): the sheet then says **Their price is not copied**, and each product waits in **Needs info**. The same happens when a store does not say which currency it uses.

While it runs

- The tile on **Products** shows what it found so far: **found · added · already here · out of stock · failed**. It refreshes by itself.
- **Cancel import** stops it between pages; products already added stay.
- Running the same store again skips every product that is **already here** (typed with or without "www"), so it is safe to repeat — for example to pick up new arrivals. The check says how many are already here before you start, and the button then reads **Check for new products**. When nothing was new the tile says **Nothing new on ...**. A repeat costs nothing: no AI is used for products that are skipped. It does not change the price or stock of the products it skips.
- An import stops by itself after **5,000** new products (products already here do not count, so running it again picks up the rest), and after three pages in a row fail — the tile then says why, with **Try again**.
- It goes at the pace of the AI: while many products are still waiting to be read or listings to be written, the import pauses, so a big store takes a day or more. If the store asks Playbook to slow down, it also waits a few minutes by itself.
- When it is done, **Show these products** lists everything it added (filtered by the vendor).
- **Settings → Imports from websites** lists the last 20 imports.

> **Tip** Sites that ask automated visitors to stay away (a `robots.txt` rule) are refused with a plain message. Playbook never works around that.

> **Note** Photos and text on another store are that store's copyright. Playbook writes new titles and descriptions with the AI for each of your websites, but the **photos are copied as they are**. Use this only on stores whose content you are allowed to reuse (your own other stores, a brand that supplies you, a store with written permission), or replace the photos before publishing.

12 Tags: your own lists

You build the tags by hand, once, on **Settings → Tags**: each **category** uses some **attributes** (Colour, Work, Fabric...), and each attribute has its **values** (Black, Yellow, White...). Playbook only

ever picks from your lists — it never invents a tag — so the tags stay tidy however many products arrive.

1 · Create attributes and values

1. Open **Settings → Categories & tags**. Tap **New attribute**, name it (e.g. `Colour`) and choose **Several** (a product can be Pink and Wine) or **One only** (one Fabric).
2. Under the attribute, type a value in **Add a value** and tap **Add** — `Black`, `Yellow`, `White`. Add **Work** (`Embroidery`, `Printed`, `Sequins`) and **Fabric** (`Organza`, `Chiffon`, `Silk`) the same way.
3. Tap a value to rename it, add the words it is **also known as** (`kala`, `kaala` for Black), **merge** it into another value, or delete it. The number on each value is how many products carry it.

2 · Link attributes to categories

Under **Categories**, tap a category (this is the same list the AI picks a category from — it used to sit on Settings) and add the attributes it uses, in the order you like. **Every attribute of a category is a must**: a product without a value for one of them waits in **Needs info** and is not sent to websites.

Before linking, Playbook counts the products that lack it — "12 products will need a Colour" — and offers **Link and let AI decide** (links it and asks the AI to choose a Colour for those 12), **Link only** or **Cancel**. Unlinking frees them again.

3 · How products get their tags

STEP	WHAT HAPPENS
From the text	When a product comes in (Telegram, Import from website) and again after the AI reads it, Playbook looks for your values and their "also known as" words in the name, the store's tags, the post and the facts. `COLOUR :- 2 (Pink, Wine)` gives Colour: Pink and Wine; `embroidery-work` gives Work: Embroidery; `Printed` and `print` count as the same word.
Then the AI	In the same read, the AI chooses from your lists only for the attributes the text left empty — the closest value when the photos or text support it (rani pink → Pink), never a word outside the list.
Still empty?	The product stays in **Needs info** with "Missing: Colour". Tap **Let AI decide** on it, or choose yourself. On **Products → Needs info**, **Let AI decide for N products** does them all in the background.

4 · Suggestions and the look-alike guard

- **Suggestions** collects words seen on products that are not in your lists yet — store tags such as `Kurtas & Suits`, words in posts, things the AI noticed — most seen first, with three example products each. Store codes like `_altnew_KBSUIT25` and `SIZE_CHART` never show up there.
- **+ Add to Colour** makes it a value in one tap; **Also known as…** files it under a value you already have; **Ignore** hides it for good. Tick several to add or ignore them together.
- Whenever a new word looks like an existing value — `Embroided` and `Embroidery`, `Sequin` and `Sequins`, `kala` and Black, `Rani Pink` and Pink — Playbook asks first: **Same thing — add as also**

known as or **It's different — add as new**. A word that already belongs to another attribute (`Silk` under Work) gets "This looks like a Fabric value".

- The same check runs as you type in **Add a value**: the closest match shows under the field.

5 · On a product

In **Facts → Tags** the values show by attribute: "Colour: Black · Work: Embroidery". Tap an attribute to change it; a tag shows **Auto** (found in the text), **AI** or **You**. What you choose is kept — the text rules and the AI never change it again. Choosing the last missing value makes the product **Ready** and sends it to its websites.

6 · On websites

A website's **Tags** section sends the values with `{tags}`: **Plain** (`Black`) or **Prefixed** (`Colour_Black`, using each attribute's Shopify prefix). New websites start with `{tags}` in their templates; **At most, per product** caps the list.

> **Tip** After adding values or "also known as" words, tap **Retag all products** on the Tags page: it first says how many products would change and how many would move between Ready and Needs info, then asks.

13 Daily use — adding a product

1. Tap **+ Add product** (on **Products** or on the **Dashboard**).
2. **Photos** — add one or more (JPG, PNG or WEBP). The first photo is the main one.
3. **Vendor** — pick one, or create a new vendor right there.
4. **Cost price (₹)** — the vendor's price to you.
5. **SKU** — optional, your product code.
6. **Details** — paste the vendor's whole post: fabric, colour, sizes, work, price.
7. **Category** and **Sizes** — optional. Leave **Let the AI decide** and the AI fills them in.
8. Tap **Add product**. The product opens; within a minute the AI reads it and the websites pick it up.

- You need at least a photo, the details text or a SKU.
- Whatever you type yourself is kept exactly as you wrote it — the AI only fills empty boxes.

> **Tip** The better the post you paste, the better the listing. Fabric, colour and sizes in the text help the AI more than the photo alone.

14 What each status means

Product status

STATUS	WHAT IT MEANS	WHAT YOU DO
New	Waiting for the AI to read it	Nothing — within a minute
Reading	The AI is reading it now	Wait; editing is off for a moment
Needs info	A necessary detail is missing: cost price, category, fabric, colour, a photo or a required tag ("Missing: Fabric, Photo") — or the AI could not read it after 3 tries (red "AI reader" note), or it looks already added ("Already here as KB 302" — see **Already added?**). Nothing is written for the websites until it is filled or answered	Open it, fill what the product says is missing in **Facts**, tap **Save facts**; add a photo with **Add photos**; for a missing tag tap it under **Tags** or **Let AI decide** — then it becomes Ready and is sent to the websites. All filled? Tap **Mark ready**
Ready	Read, and checked against every website's rules	Nothing. See where it went in the **Websites** box
Archived	Hidden from the list and not sent anywhere	**Restore** brings it back

Listing status (one product on one website)

YOU SEE	INTERNAL NAME	WHAT IT MEANS	WHAT YOU DO
Queued	`queued`	Waiting to be written	Nothing
Writing	`generating`	The AI is writing title, description, SEO and tags	Wait; editing is off
Needs review	`review`	Written; this website is on **Review first** — or a check found something ("Check before it goes out: ...")	Open, read the reason, edit or **Regenerate**, then **Approve**
Publishing	`publishing`	Being sent to Shopify	Wait; editing is off
Published	`published`	On Shopify	**Shopify** opens it in the Shopify admin
Failed	`failed`	Shopify or the AI refused; the red text says why	See "Fixing a failed listing"
Removed	`skipped`	Taken off this website (Draft on Shopify)	**Put back on site** if needed

In the **Products** status filter the listing statuses start with "Listing:". **Listing: In progress** means Queued, Writing and Publishing together.

15 Working with listings

Open a product, then tap a website in its **Websites** box. The listing opens with its price, title, description, SEO, handle and tags, and the buttons below.

Fixing a failed listing

1. On the **Dashboard** tap **Failed to publish** (or filter Products by **Listing: Failed**).
2. Open the product, then the website shown with the red text.
3. Read the red text and fix the cause — see **Troubleshooting**.
4. Tap **Publish now** (or **Update on Shopify** if it is already there).

The app also tries again by itself — up to 3 times, 10 minutes apart — so a short Shopify hiccup clears on its own.

Editing, and "kept" fields

Change any of **Price (₹)**, **Title**, **Description**, **SEO title**, **SEO description**, **Handle (URL)** and **Tags (comma list)**, then tap **Save changes**. Each field you change gets a small **kept** label: **Regenerate** will never touch it. Product **Facts** work the same way — a box you typed is kept and the AI reader never overwrites it.

> **Tip** To hand a field back to the AI: empty the box, tap **Save changes**, then **Regenerate**. The **kept** label goes, and from then on the AI (and, for the price, the cost and the website's pricing) decides that field again. This works for price, title, description and both SEO fields.

- On a published listing, after **Save changes** tap **Update on Shopify** to send the change.
- While a listing is being written or sent to Shopify, editing is off. Close it and open it again in a minute.
- Once a product is on Shopify its web address (handle) never changes, so old links keep working.

The buttons

BUTTON	WHAT IT DOES
Regenerate	Writes the listing again with the website's current settings and works the price out again. Kept fields stay. On a Live or Draft website it is then sent to Shopify by itself within a minute; on Review first it waits for Approve.
Approve	Review first websites: sends the listing to Shopify within a minute. While **Automation** is paused it waits — tap **Publish now** to send it. A website that is switched off cannot approve.
Publish now / **Update on Shopify**	Sends it to Shopify straight away instead of waiting.
Remove from site	Sets the product to Draft on Shopify — hidden from shoppers, not deleted. Status **Removed**. It is not re-listed on its own.

BUTTON	WHAT IT DOES
Put back on site	Sends a removed listing to Shopify again. One removed before the AI had written it is written first, then sent.
Open in Shopify	Opens the product in that store's Shopify admin.

On the product itself

BUTTON	WHAT IT DOES
Read again with AI	The AI reads the photos and text again and refreshes every field it filled. Kept fields stay.
Check websites again	Runs every website's rules again for this product — use it after you change a rule. Published listings also get the price from today's cost and pricing, sent to Shopify without rewriting the text (a **kept** price stays).
Save facts	Saves your changes. If the cost, category, vendor, fabric, name, SKU or details changed, the websites are checked again: unpublished listings are re-priced and rewritten, published ones are re-priced and updated on Shopify (their text stays; a **kept** price stays).
Archive / **Restore**	Archive hides the product and stops sending it anywhere. Listings already on Shopify stay — use **Remove from site** first if they should go. Restore makes the AI read it again.
Delete	Admins and managers only (the bin button; on a phone under **More actions**). After one confirmation the product is removed from Playbook completely: facts, photos, tags, listings and history. It cannot be undone. Not possible while the product is on Shopify — tap **Remove from site** on that website first. A product copied from a website is added again by the next import of that website; **Archive** keeps it out.
+ Add photos	Adds more photos to the product.

16 Already added? — copies, colours and where a product came from

The same design often arrives twice: in a vendor's Telegram post and again from a website, or in two posts. Playbook compares every product with the ones already here — by its design code, and by its photos — so one design is not listed twice.

What it does by itself

IT FINDS	WHAT HAPPENS	WHAT YOU DO
The very same post or page again (the same Telegram message, the same page of a website, typed with or without "www")	It is not added a second time. An import counts it as **already here**	Nothing

IT FINDS	WHAT HAPPENS	WHAT YOU DO
The same design code from the **same vendor**, in the same colour or with no colour given (`KB 302` and `KB 302 XS Blue`)	The later one waits in **Needs info · Already here as KB 302 XS Blue**. Nothing is written for the websites	Open it and answer (below)
The same code from the same vendor in a **different colour** (`KB 264 Wine` and `KB 264 Royal Blue`)	Nothing waits: each colour is its own product. The product page lists the other colours under **Same design**	Nothing
A near match: the same code at **another vendor**, or the same number with other letters (`KB 25` and `NKB 25`)	Nothing waits. The product page asks **Could this be already added?**	Answer when you see it
The **same photo** on two products — saved again at another size or quality — when their codes say nothing (no code, or different codes)	Nothing waits. The product page asks **Could this be already added?** and shows the photo that matched, marked **Same photo**	Answer when you see it

How the code is read

- From the **SKU**: letters, then numbers. `KB 302` , `KB-302` , `kb302` and `KB 302 XS Blue` are all the design **KB 302** — spaces, dashes and capitals do not matter, and size or colour words after the code are not part of it.
- The colour is the words after the code (`KB 264 ROYAL BLUE XS` → royal blue); when there are none, the product's **Colour**.
- A product without a SKU is never held back.

How photos are compared

- Each photo is checked once, in the background, a few minutes after the product arrives. No AI is used and it costs nothing.
- It recognises the **same picture** saved again: smaller, larger, more compressed, a little brighter, or turned upright from a phone. A different photo of the same dress is **not** matched, and neither is a cropped or mirrored copy.
- It does not look at colour, so the same pose in another colour can match — that is why a photo match only asks, and never holds a product by itself.
- A picture that is on many products (a size chart, a banner) is ignored. JPEG photos only; PNG and WEBP photos are not compared.

Answering

A product that waits shows **This looks already added** with the earlier product (photo, code, colour, where it came from). Tap the earlier product to open it and compare, then answer:

BUTTON	WHAT IT DOES
It is a copy	Archives this product; the earlier one stays. **Restore** brings it back.
Another colour	Keeps both as separate products. This one becomes Ready and goes to the websites.
Different product	Keeps both — for two designs that happen to share a code. This one becomes Ready and goes to the websites.

- An answer is remembered: the same two products are never asked about again.
- Nothing is merged or deleted, on either product.
- Tapped the wrong answer? **Archive** the copy yourself.

Where a product came from

- Each product on **Products** says where it came from: "from Telegram" or "from bahupalace.com". A product you typed in shows nothing.
- The **Source** filter lists only the products from Telegram, typed in, or copied from one website.

> **Tip** Give vendors' products their design code as the SKU (the AI reads it from the post). Without a code only the photos can be compared.

17 Troubleshooting

WHAT YOU SEE	LIKELY REASON	WHAT TO DO
Test connection fails, or a red "Last problem"	Wrong or old token / secret, the app was uninstalled, or the store domain is not the myshopify.com one	Copy the details again from Shopify, **Save credentials**, **Test connection**
"Missing permissions: ..."	The app in Shopify does not have every scope	Add the listed scopes to the app, release / reinstall it, test again
Client ID + secret refused although they are right	The app is in a different Shopify organisation, or not installed on this store	Create the app in the store's own organisation and install it on the store
Active will not switch on	The store has never passed a test	**Save credentials**, then a green **Test connection**
Products stay **New** for more than a few minutes	Automation is paused, or a big batch is being worked through	Dashboard banner → **Open Settings** → switch Automation on; or tap **Run now**
Delete says "This product is on Shopify (...)"	A listing of it is on that website and was not taken off	Open that website on the product → **Remove from site** → then **Delete**

WHAT YOU SEE	LIKELY REASON	WHAT TO DO
Product in **Needs info**	Cost price, category, fabric, colour, a photo or a required tag missing ("Missing: Fabric, Photo"), or the AI could not read it (red "AI reader" note)	Fill the facts → **Save facts** (already filled: **Mark ready**); a photo: **Add photos**; a tag: **Let AI decide** or choose it under **Tags**; or **Read again with AI**
Needs info · Already here as ...	The same design code from the same vendor is already in Products	Open it; under **This looks already added** tap **It is a copy**, **Another colour** or **Different product**
Listing in **Needs review** on a **Live** website	A check found something: copied wording, the search phrase missing, the same title as another store... The listing says which	Open it, read "Check before it goes out", fix the text or tap **Regenerate**, then **Approve**
Product **Ready** but on no website	No rule matched and the website is on **Do not list it**; or the website is Off	Check the rules (conditions, order, switches) and **Active**, then **Check websites again**
Product went to the wrong website	A wide rule matched first	Move a "Keep it off" rule above it; **Remove from site** on the wrong one
Failed: "The AI answer was cut off — try a lower effort setting"	Effort too high for the length limits	**Content & AI** → lower **Effort** → **Save changes** → **Regenerate**
Published, but shoppers cannot see it	The website is on **Draft**; or the publications permission is missing (noted in the listing's red text)	Make it visible in Shopify, or add `read_publications` + `write_publications` and tap **Update on Shopify**
The price is not what you expected	Markup, rounding, minimum or a rule's own price — or the price is **kept**	Try the cost in the **Pricing → Preview** box; check the rule; empty the price box → **Save changes** → **Regenerate**
A tag is missing	Its placeholder is empty for this product (e.g. no fabric)	Fill that fact on the product → **Save facts** → **Regenerate**
Listing boxes are greyed out	It is being written or sent to Shopify right now	Close it and open it again in a minute
"Product saved, but the photos did not upload"	Weak internet, or the file type	On the product tap **+ Add photos** (JPG, PNG or WEBP)

18 Questions people ask

Does the app ever delete a product from Shopify?

No. **Remove from site** only sets it to Draft. **Archive** and **Delete** do not touch Shopify at all — and Delete is refused while the product is still on Shopify.

I changed a website's pricing / tags / writing. Are old products updated?

Not by themselves. New listings use the new settings. For a new **price** on a product already on Shopify, open the product and tap **Check websites again** — the price is updated on Shopify and the text stays. For new tags or writing, open the listing and tap **Regenerate** (and **Update on Shopify** if it does not go by itself).

Can one product have a different price on each website?

Yes. Every website has its own formula, and a rule can give a group of products its own markup.

The vendor's post shows a retail price, not our cost. What then?

The app always treats the number as your cost. Type your real cost in **Cost price (₹)** — what you type is kept.

Will the AI make things up?

It is told to use only the facts given and never to invent a fabric or size. Read the first listings of each website; anything you correct is kept.

What does the AI cost?

One AI call reads each product, and one call writes each listing on each website. Haiku is the cheapest; use Sonnet or Opus only where the writing really matters.

Are prices with or without GST?

The app sends the number from your formula. Whether Shopify treats prices as including tax is set in Shopify (Settings → Taxes and duties).

Who can see the Shopify keys?

Nobody. They are stored on the server, used only to talk to Shopify, and never shown again in the app.

What happens when I pause automation?

Nothing runs on its own until you switch it back on; buttons on a product still work. When it is switched on again it carries on from where it stopped.

Two of us edited the same listing. Whose change stays?

The last one saved. Close and reopen a listing before editing if someone else may have just changed it.